



6 / 10 / 2026

SQA PROJECT

PRESENTATION

KHONKAEN UNIVERSITY

Datasets and Tools

1

DEFECT4j Dataset

854

ACTIVE BUGS

17

PROJECT

Algorithm Based

UTBot

TARDIS

Generative AI

Claude Sonnet 5

Gemini 3.5 & 3.7

Evaluation Metrics

- **Executability**

Test ที่สร้างมาสามารถ Compile และ รันได้จริงแค่ไหน

- **Fault Detection**

Test สามารถตรวจจับ Bug ได้จริงที่ รายการ

- **Code Coverage**

Test สามารถเข้าถึง Source Code ได้ ครอบคลุมแค่ไหน

- **Runtime/Token Usage**

ใช้เวลา หรือ Token ในการสร้าง Test มากแค่ไหน

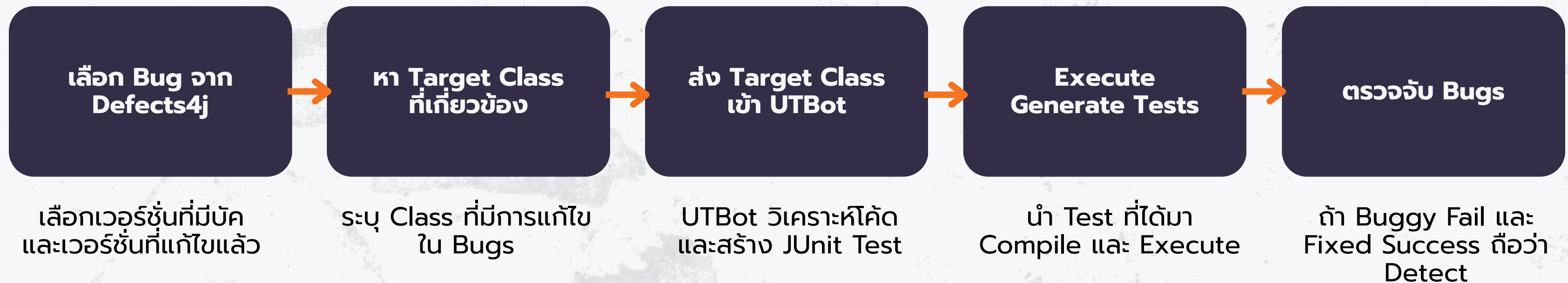
เป็นเครื่องมือที่เน้นยิง Input รัวๆ เข้าใส่ระบบเพื่อบังคับให้โค้ดทำงาน โดยใช้ เทคนิคการ Fuzzing และ Symbolic Execution โดยกระบวนการทำงานของ UTBot ประกอบด้วย



UTBot : Work Flow

3

กระบวนการทำงาน และการตรวจวัดความสามารถในการตรวจจับ Bugs ของ UTBot โดยมีขั้นตอนการทำงานดังต่อไปนี้



UTBot : Result

4

ผลการ Execute, Fault Detection และ Line Coverage ของ UTBot แยกตาม 17 Project

Project	Executable	Detected	Project	Executable	Detected
Chart	1 / 26	0	JacksonData	3 / 110	0
Cli	4 / 39	1	JacksonXML	0 / 6	0
Closure	6 / 174	0	JSoup	8 / 93	0
Codec	4 / 18	2	JXPath	1 / 22	0
Collections	0 / 28	0	Lang	15 / 61	1
Compress	8 / 47	1	Math	68 / 106	2
Csv	0 / 16	0	Mockito	5 / 38	0
Gson	5 / 18	0	Time	11 / 26	0
JacksonCore	0 / 26	0			

Math Execute สำเร็จสูง เพราะ Generated Tests ของ UTBot เข้ากับ Environment ของ Math ได้ดี ทำให้ได้ 68 จาก 106 Bugs (64.15%) ที่รันสำเร็จ แต่ตรวจจับ Bug ได้จริงเพียง 2 Bugs

UTBot : Result

5

139

Executable Bugs

7

Kill Bugs

53.24%

Line Coverage
(UTBot Only)

45.75%

Condition Coverage
(UTBot Only)

Runtime & Resource Usage

Total Generation Time: 43.23 ชั่วโมง

Average Generation Time: 2.42 นาที / Class

Generation Attempts: 1,073 Classes

Resource: ใช้ Local CPU และ RAM ในการ
Generate Test

Coverage Boost (UTBot + Developer)

UTBot Line Coverage: 53.24%

UTBot Condition Coverage: 45.75%

Developer Line Coverage: 89.25%

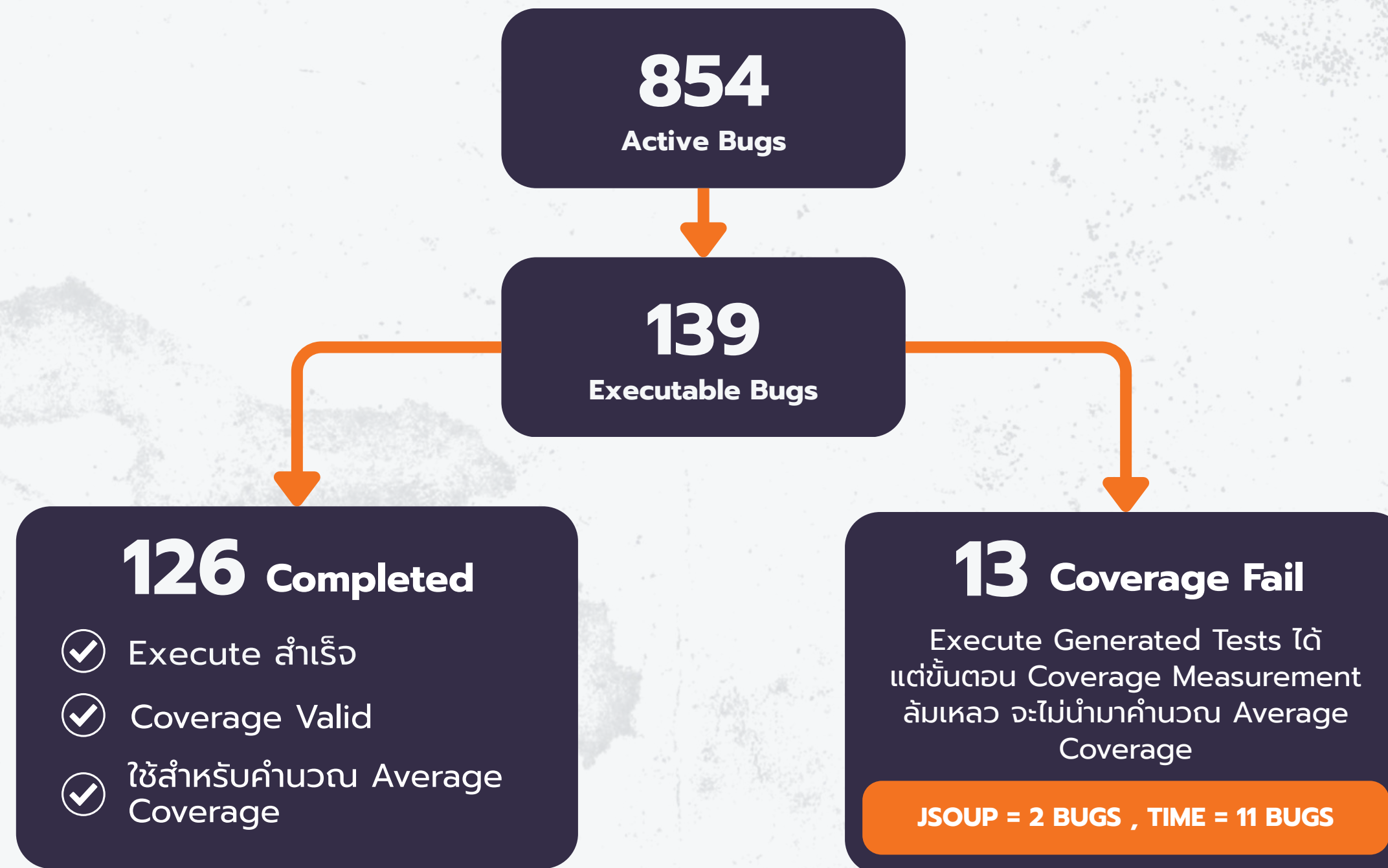
Developer Condition Coverage: 83.27%

Combine Line Coverage: 92.41%

Combine Condition Coverage: 87.17%

UTBot : Coverage Validity

6





TARDIS

METAHEURISTICALLY-DRIVEN DYNAMIC
SYMBOLIC EXECUTION

INTRODUCTION

เป็นเครื่องมือวิเคราะห์โค้ดเชิงลึกทางคณิตศาสตร์ เพื่อหา Edge Case (กรณีสุดวิสัยที่มนุษย์นึกไม่ถึง) ใช้เทคนิคที่เรียกว่า Concolic Execution (Symbolic + Concrete) และอีกเทคนิค คือ EvoSuite





Operating System :

- Linux (Ubuntu 20.04 / 22.04 LTS หรือ Windows 11 ผ่าน WSL2)

Software Dependencies :

- PYTHON 3.8+
- OPENJDK 8 หรือ OPENJDK 11
- DOCKER ENGINE & DOCKER CLI
- GIT, PERL (สำหรับรับ DEFECTS4J)

TOTAL BUGS

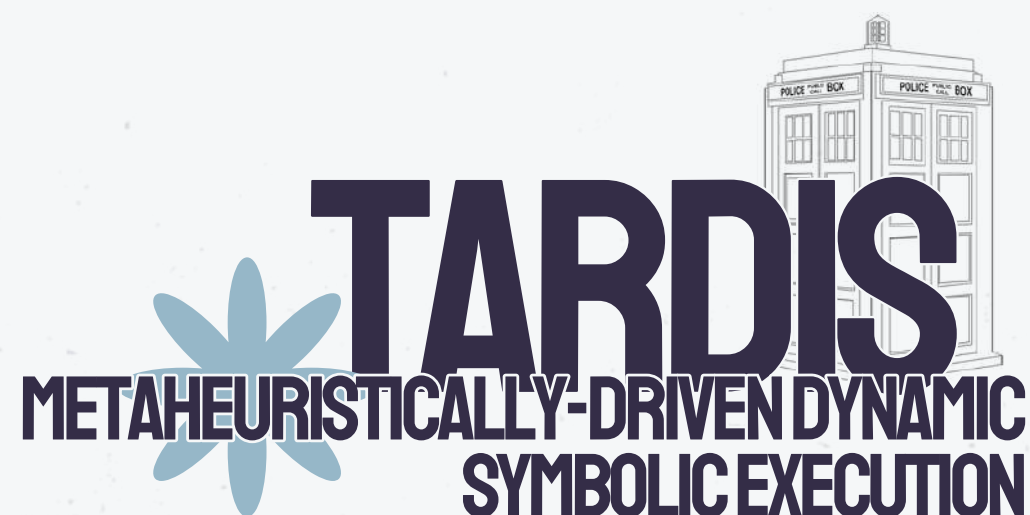
854

EXECUTED BUG

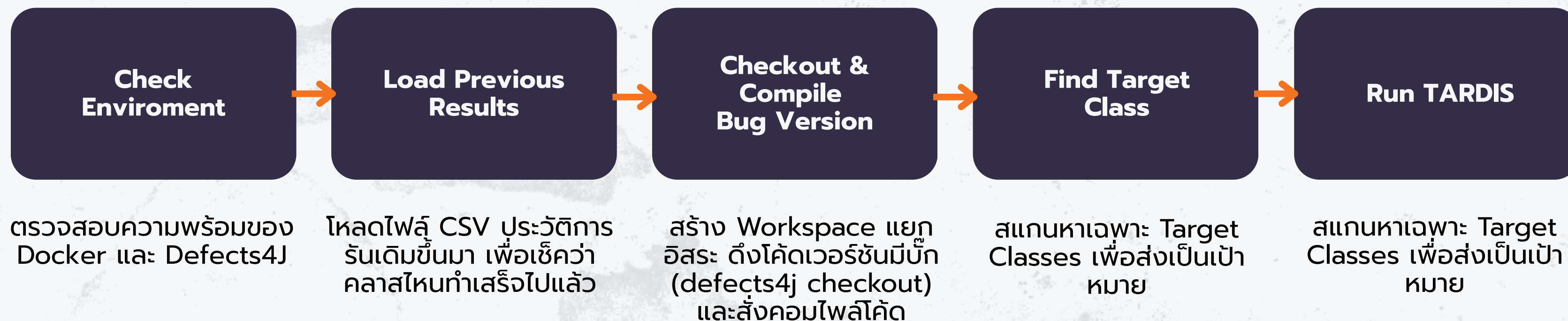
545

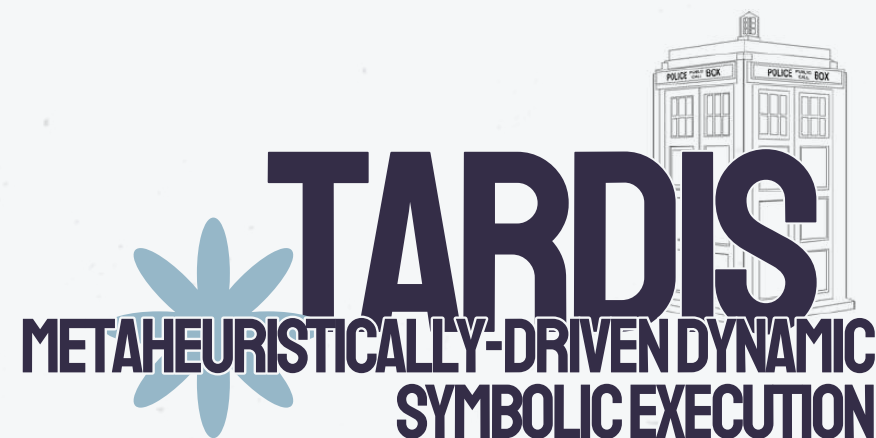
DETECTED BUG

16



RUNNER PIPELINE



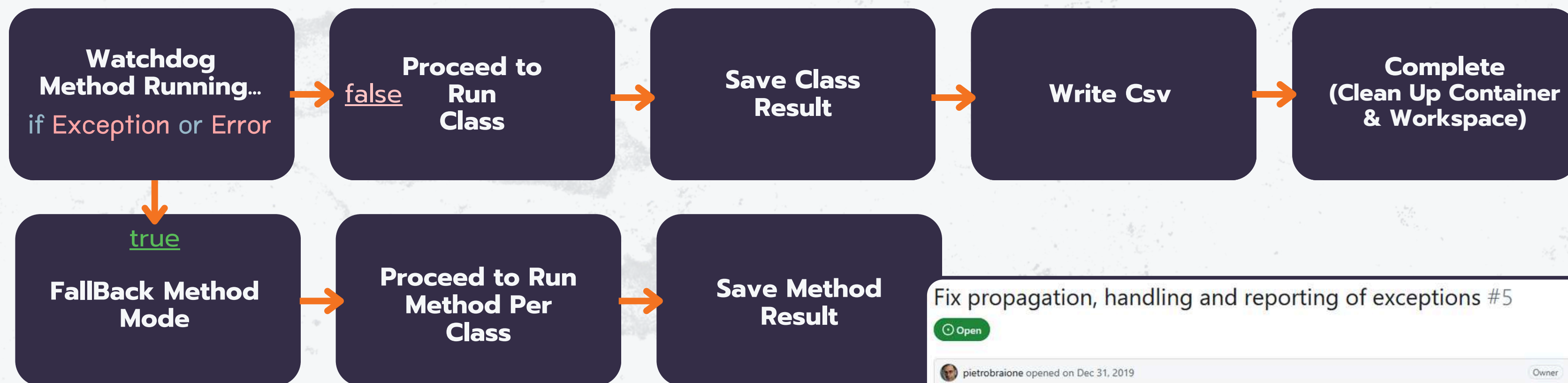


(HANDLING)

RUNNER PIPELINE

10

ในปัจจุบัน Tardis ยังมีบัค และไม่มีระบบรองรับหาก Algorithm เกิด Exception จะไม่ถูก report กลับมา และทำให้ตัว Tardis ค้างโดยไม่แจ้งเตือนอะไร เราจึงต้องแก้ปัญหา โดยสร้างสคริปต์ ซึ่งจะทำตัวเป็น Watchdog ถ้าระบบจับได้ว่า Docker Failed/ Crashed /Exception /MethodNotFoundException เกิดขึ้นหรือไม่



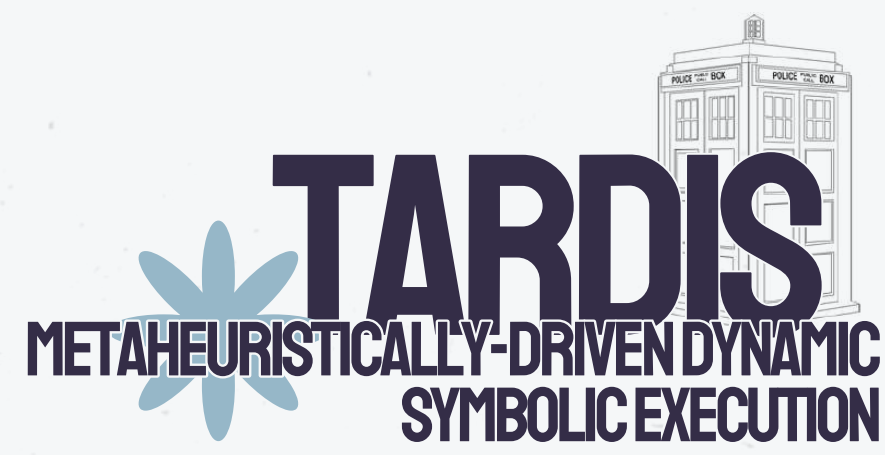
Fix propagation, handling and reporting of exceptions #5

Open

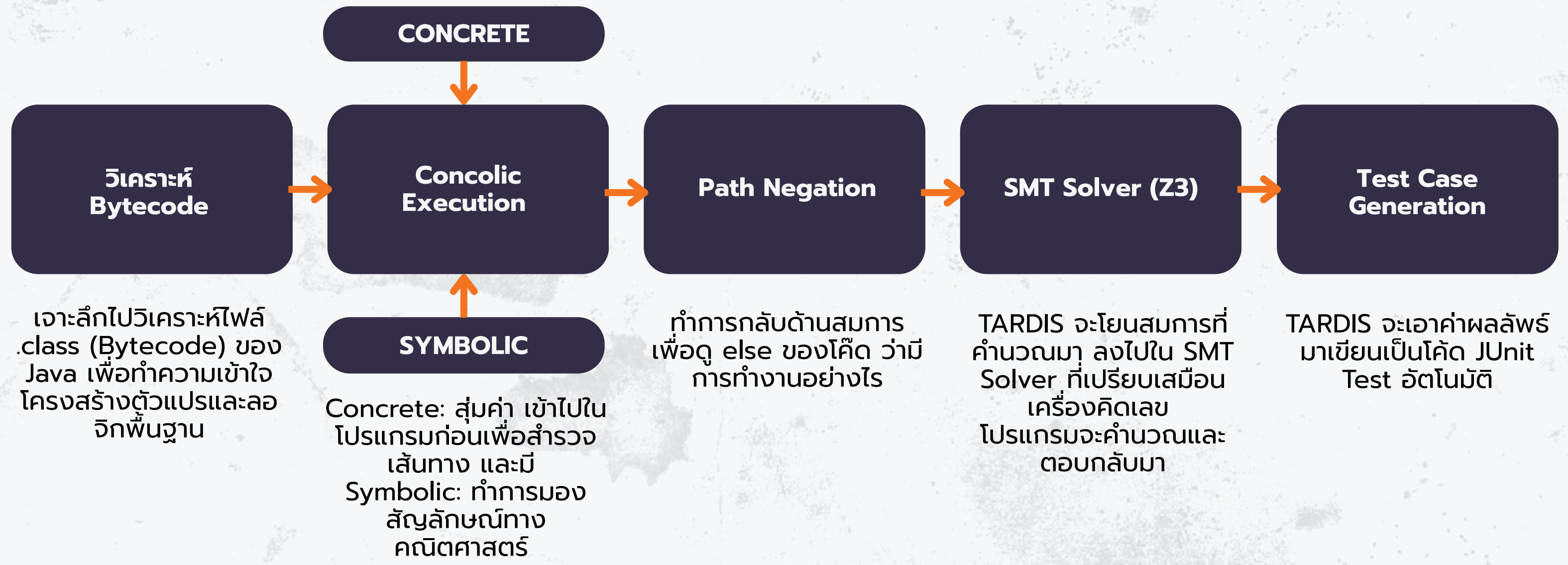
pietrobraione opened on Dec 31, 2019

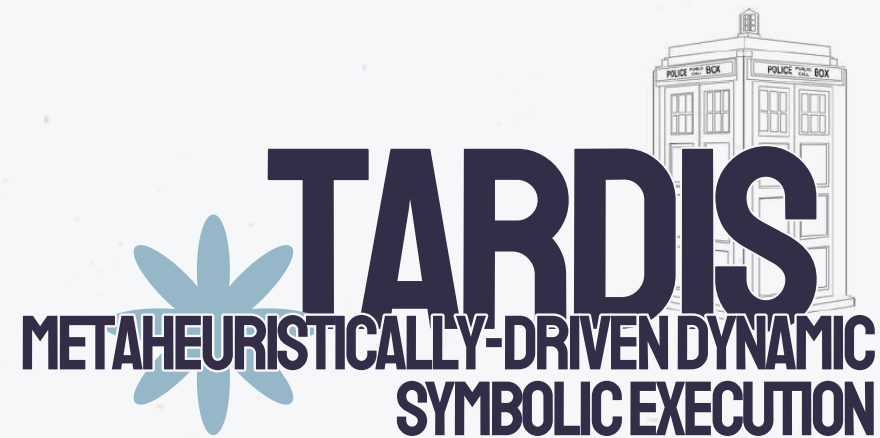
Owner

Right now exception are badly managed. They are typically caught close to their source, reported to the console and not propagated. We should distinguish between the exception that generate warnings, exception that somehow affect the workflow, and exception that abort TARDIS (if any), and properly handle them. This is not so simple given the multithreaded architecture of TARDIS, so some study on how exception propagation mixes with the executor framework is required before attempting such an endeavour.



ALGORITHM PIPELINE





RESULT

545

Executable Bugs

16

Kill Bugs

15.7%

Line Coverage
(n = 545)

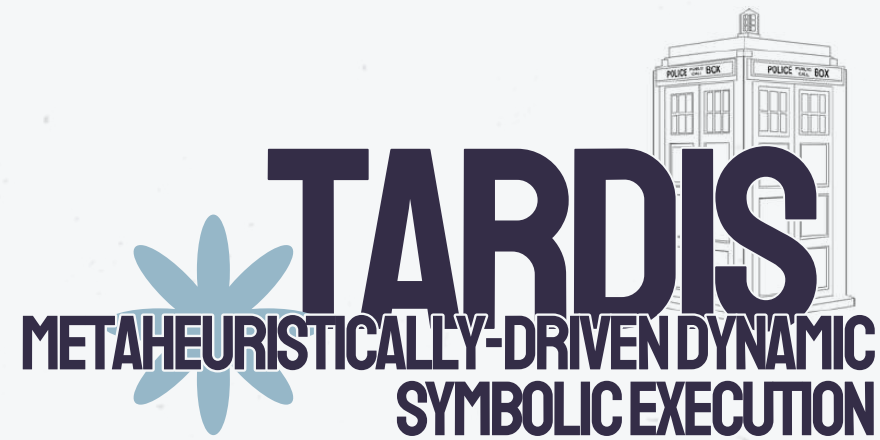
6.8%

Branch Coverage
(n = 545)

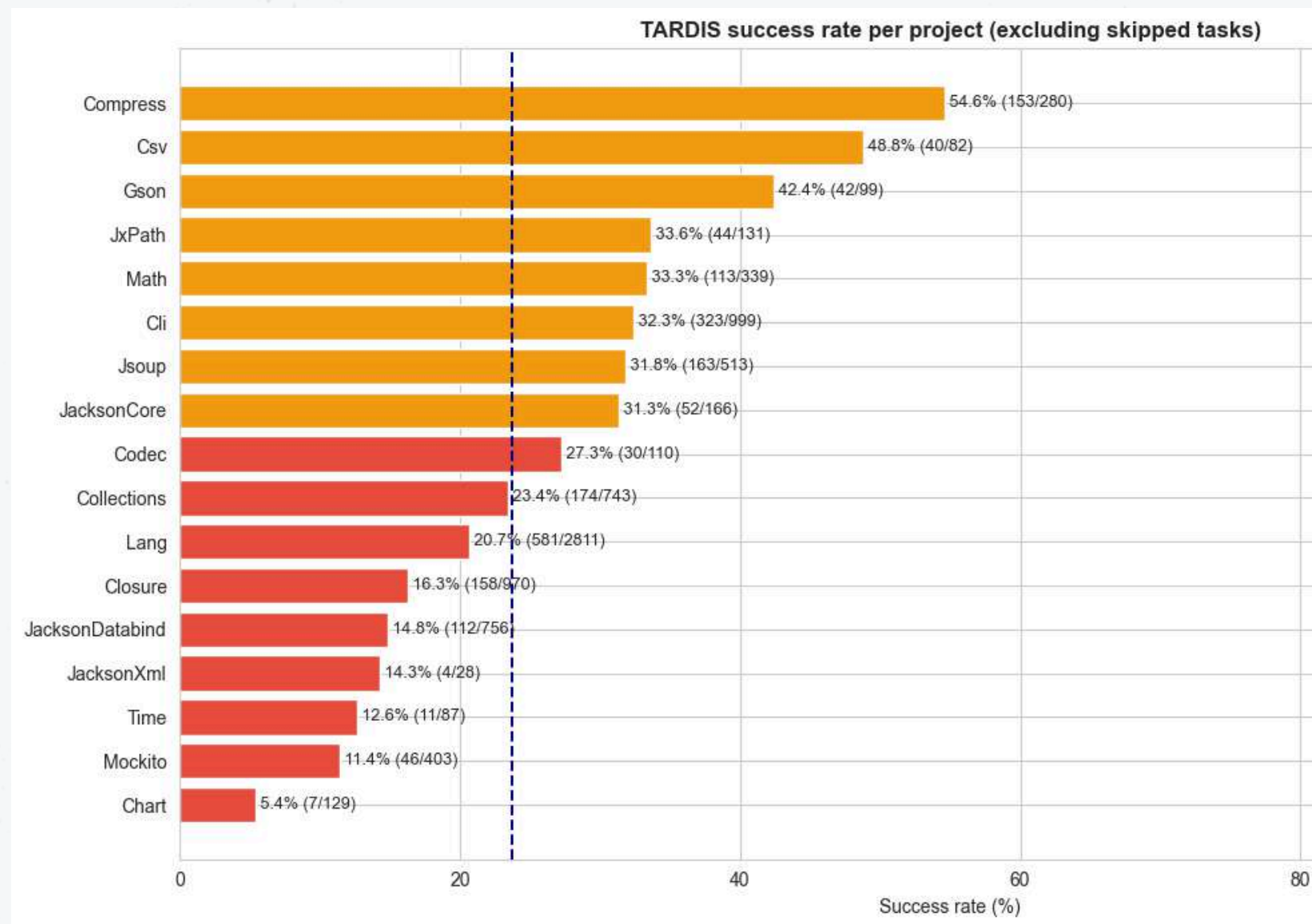
Runtime & Resource Usage

Total Generation Time	: 87.25 ชั่วโมง
Average Generation Time	: 367.8 วินาที / Class (n=854)
Timeout Generation Time	: 120 วินาที
Generation Attempts	: 8,661 Classes
Resource	: 10 gb of ram (Parallel at 5 + 5gb)

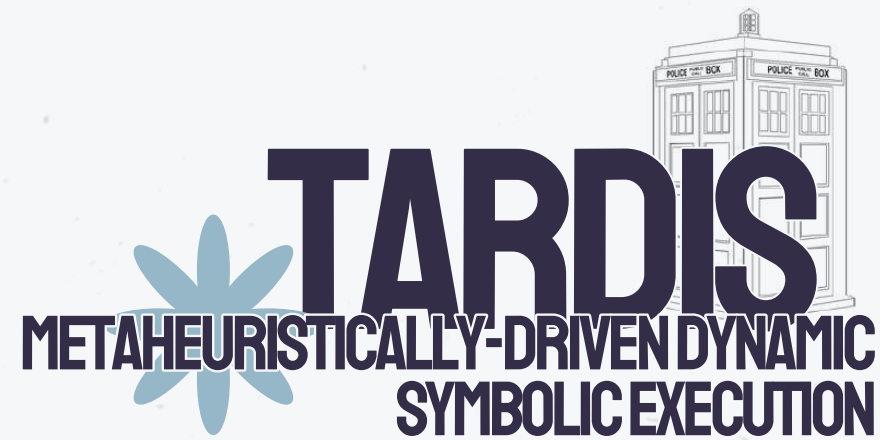
มี สมมุติฐานว่า หากเพิ่มเวลา TIMEOUT ก็จะขยาย LINE COVERAGE และ KILL BUGS RATE อย่างมีนัยสำคัญ



RESULT

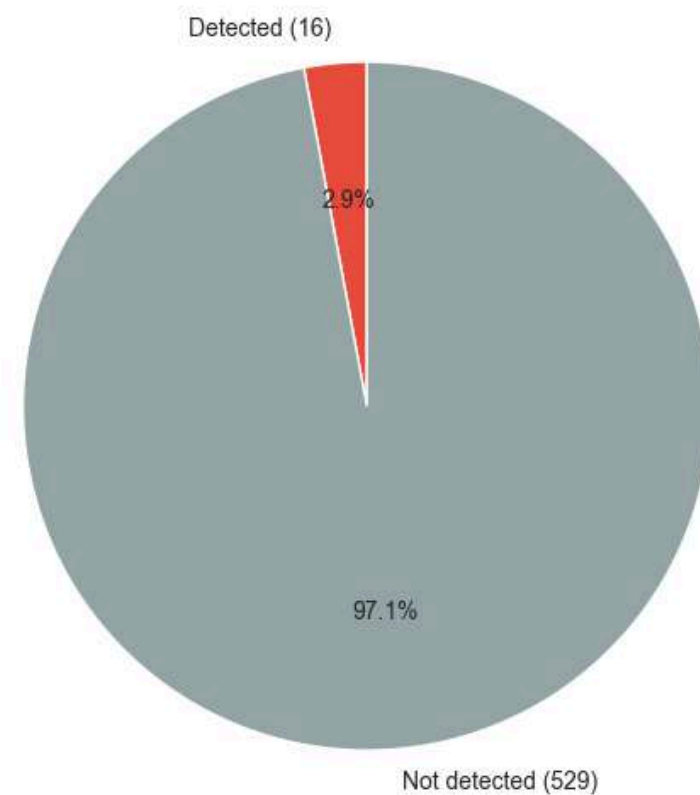


Compress เป็นโปรเจกต์แบบ Standalone ไม่มี Framework หรือ Mock จากภายนอก ทำให้เกิด Crash ได้ยาก และโค้ดของ Compress ก็เน้น Algorithm และ คณิตศาสตร์ ซึ่งเป็นสิ่งที่ TARDIS ถนัดและทำให้ตรงเงื่อนไขได้เร็วและแม่นยำ

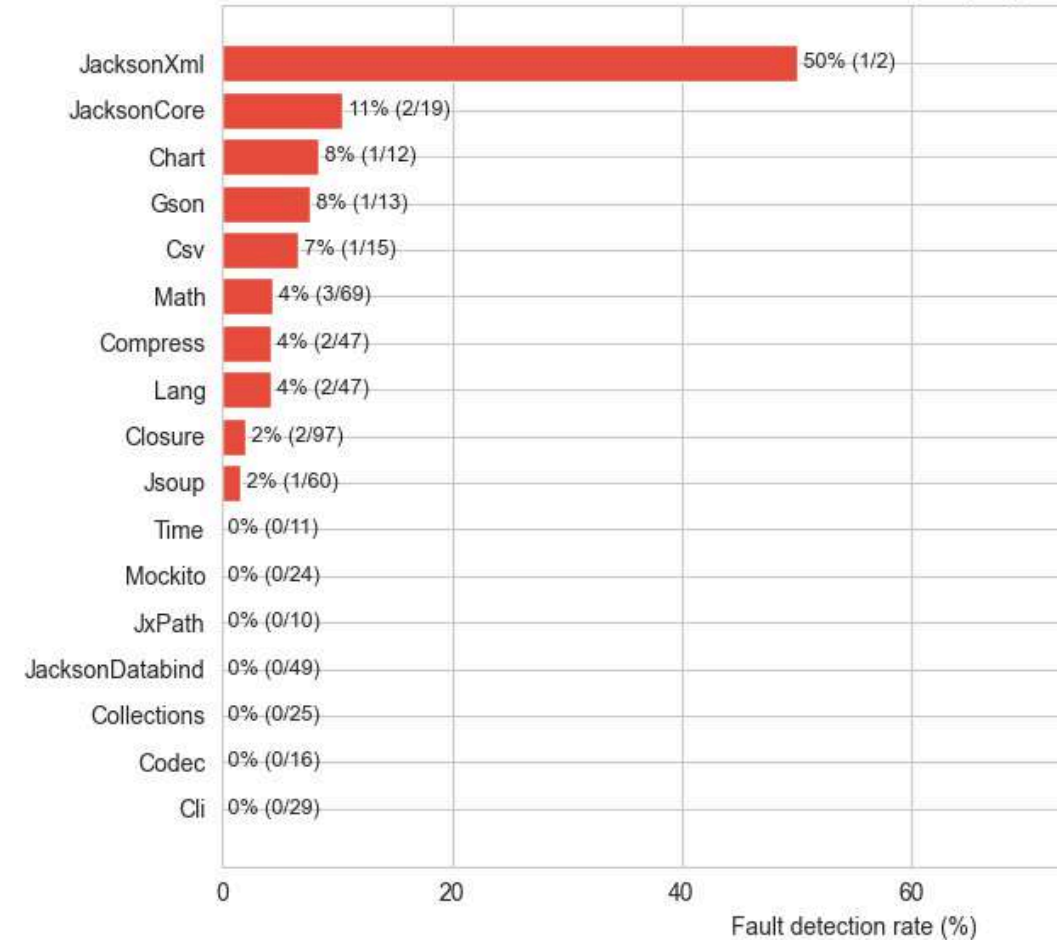


RESULT

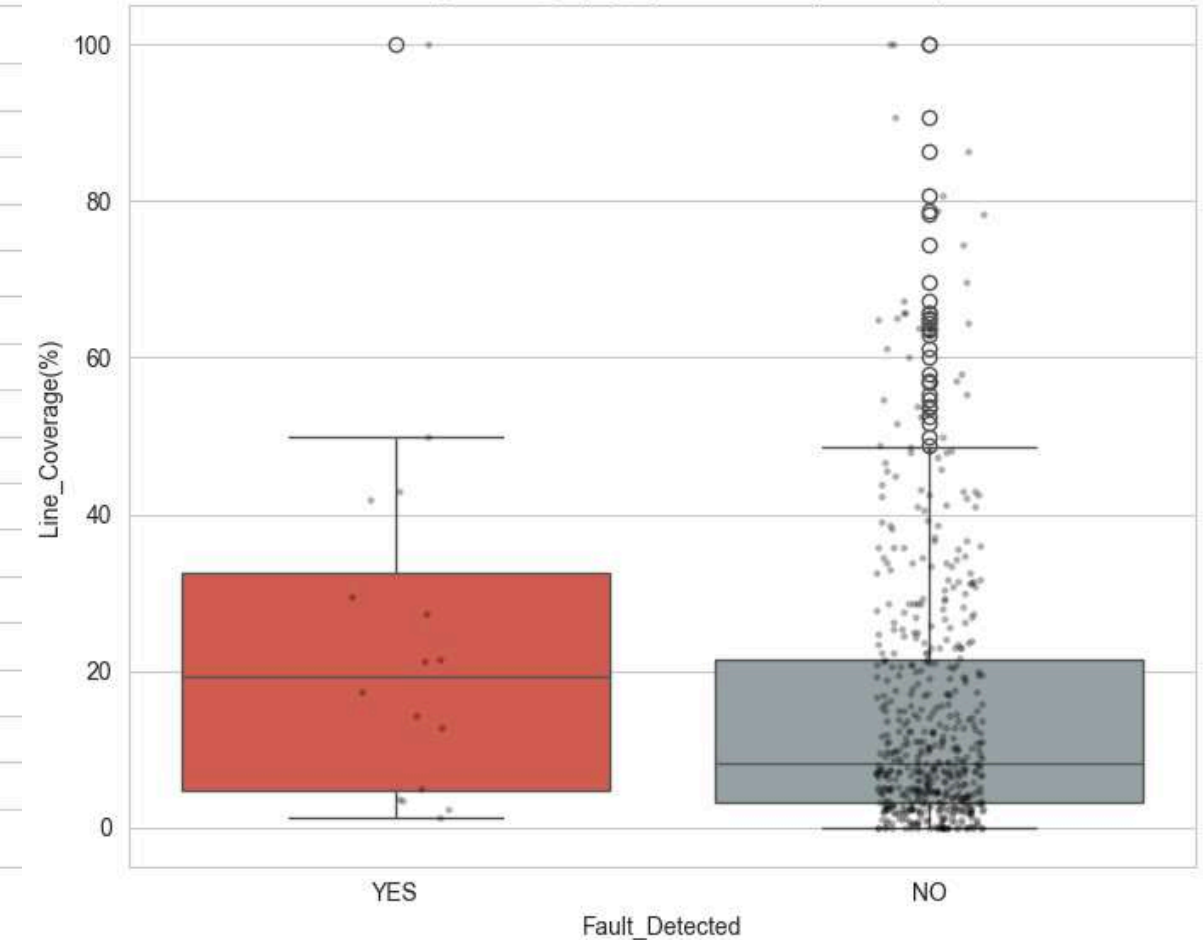
Fault detection rate (n=545 evaluated bugs)



Fault detection rate per project



Line_Coverage(%) by detection (p=0.0431)



Coverage กับ Kill Bugs: มีนัยสำคัญ ($p=0.0431 < 0.05$) , กลุ่มที่จับบักได้ มี 16 บั๊ก ค่ามัธยฐาน Line Coverage อยู่ที่ 19.2% , กลุ่มจับบักไม่ได้: มี 529 บั๊ก มีค่ามัธยฐาน Line Coverage อยู่ที่ 8.2%

GEMINI



Model

3.5 flash lite 854 တံ
3.5 flash lite 662 တံ
+ 3.7 flash 192 တံ

COMPLIE

3.5 flash lite 342/854 →
(42.86%)
3.5 flash lite + 3.7 flash
366/854 → (40.06%)

Kill_bugs

3.5 flash lite → 15 တံ
3.5 flash lite 662 တံ + 3.7
flash 192 တံ → 20 တံ

PROMPT

คุณคือ SENIOR JAVA TEST AUTOMATION ENGINEER ที่เชี่ยวชาญการหาบั๊กเชิงลึกและ JUNIT 4

เป้าหมาย: เขียนชุดทดสอบ JUNIT 4 สำหรับคลาสด้านล่าง โดยมุ่งเน้นที่การทำ BRANCH/CONDITION COVERAGE สูงที่สุดเท่าที่เป็นไปได้ และจำลองสถานการณ์ EDGE CASES เพื่อตรวจจับ FAULT ที่อาจแฝงอยู่ใน DEFECTS4J

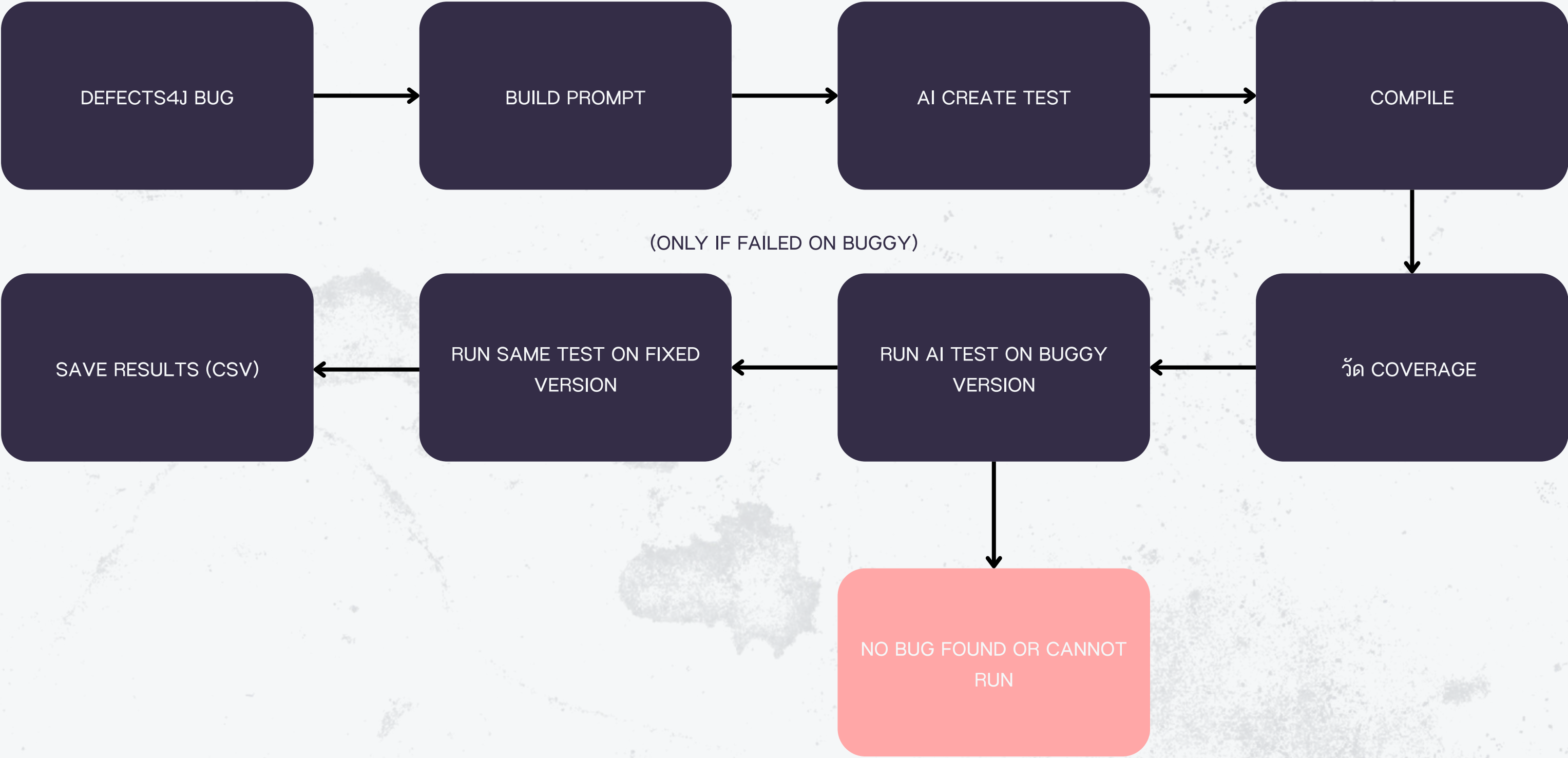
ข้อกำหนดที่ต้องปฏิบัติตามอย่างเคร่งครัด:

1. การใช้ไลบรารี: ใช้แค่ JUNIT 4 และไลบรารีที่ระบุใน CLASSPATH นี้เท่านั้น: `{CP_TEST}` (ห้ามใช้ MOCKITO หรือไลบรารีอื่นนอกเหนือจากนี้เว้นแต่จะอยู่ในซอร์สโค้ด)
2. กฎการเขียนโค้ด: ตั้งชื่อคลาสเป็น `{CLASS_NAME}TEST`
3. วิเคราะห์ก่อนเขียน (CHAIN-OF-THOUGHT): ก่อนจะเขียนโค้ด ให้คุณวิเคราะห์สั้นๆ ว่าเมรอนี้มี BRANCH อะไรบ้าง และคุณวางแผนจะใช้ อินพุตแบบไหนไป TRIGGER แต่ละ BRANCH
4. ครอบคลุมสถานการณ์: ต้องมีกรณี BOUNDARY LIMITS, NULL/EMPTY VALUES, และ INVALID STATES
5. สรุปผล: ท้ายโค้ดให้ทำตารางสรุปว่า TEST METHOD ชื่ออะไร ทำหน้าที่คลุม BRANCH ไหน

ซอร์สโค้ดคลาสเป้าหมาย (`DEFECTS4J: {PROJECT}-{BUG_ID}B`):

```
```JAVA
{SOURCE_CODE}
```
```

WORKFLOW

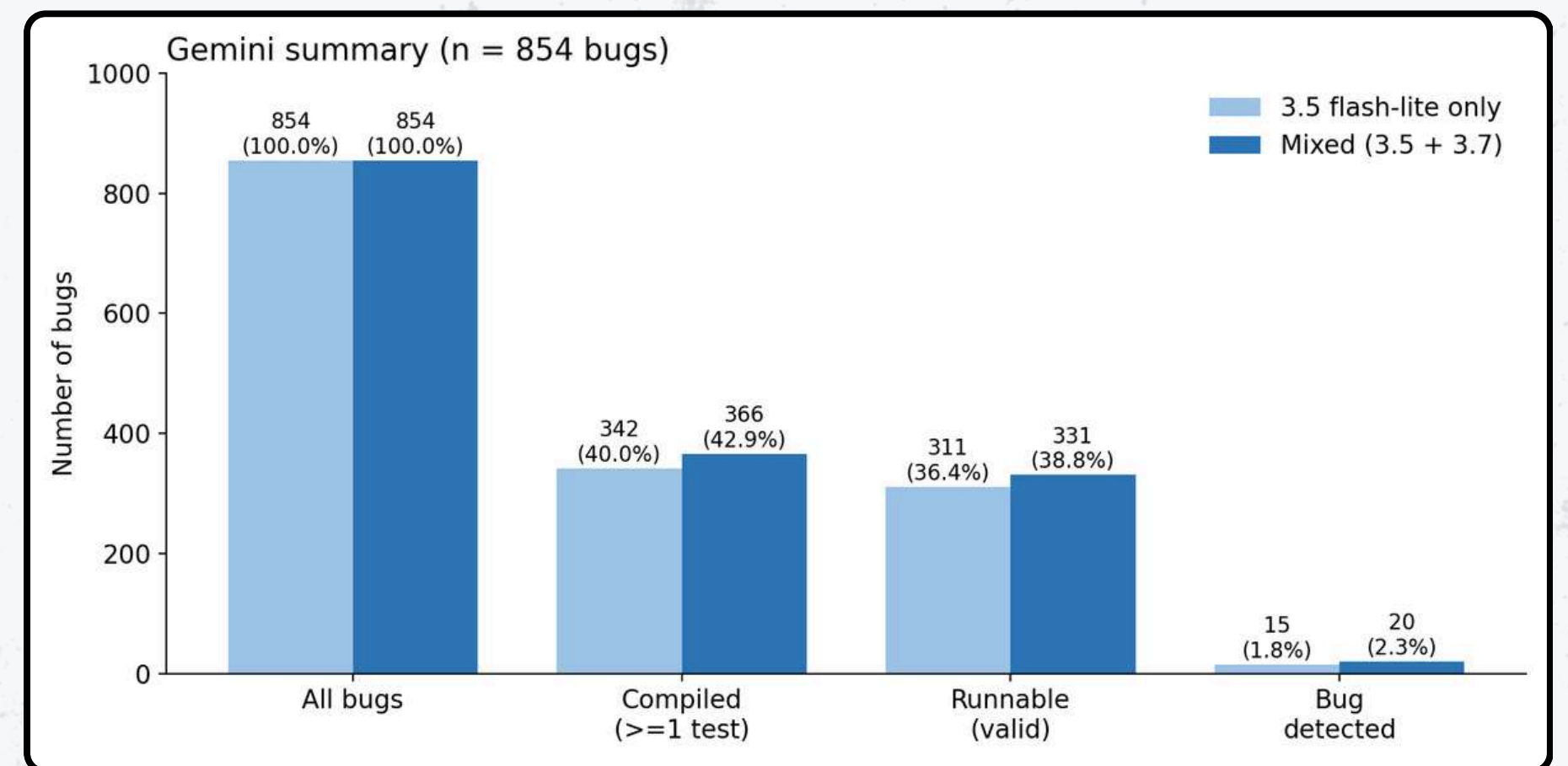


RESULTS

18

| | ผสม (3.5 + 3.7) | flash-lite ล้วน (3.5) |
|--------------|-----------------|-----------------------|
| จับได้ | 20 | 15 |
| FDR | 2.34% | 1.76% |
| compile ผ่าน | 366 (42.86%) | 342 (40.05%) |
| Coverage | 91.33% / 85.41% | 90.46% / 84.02% |

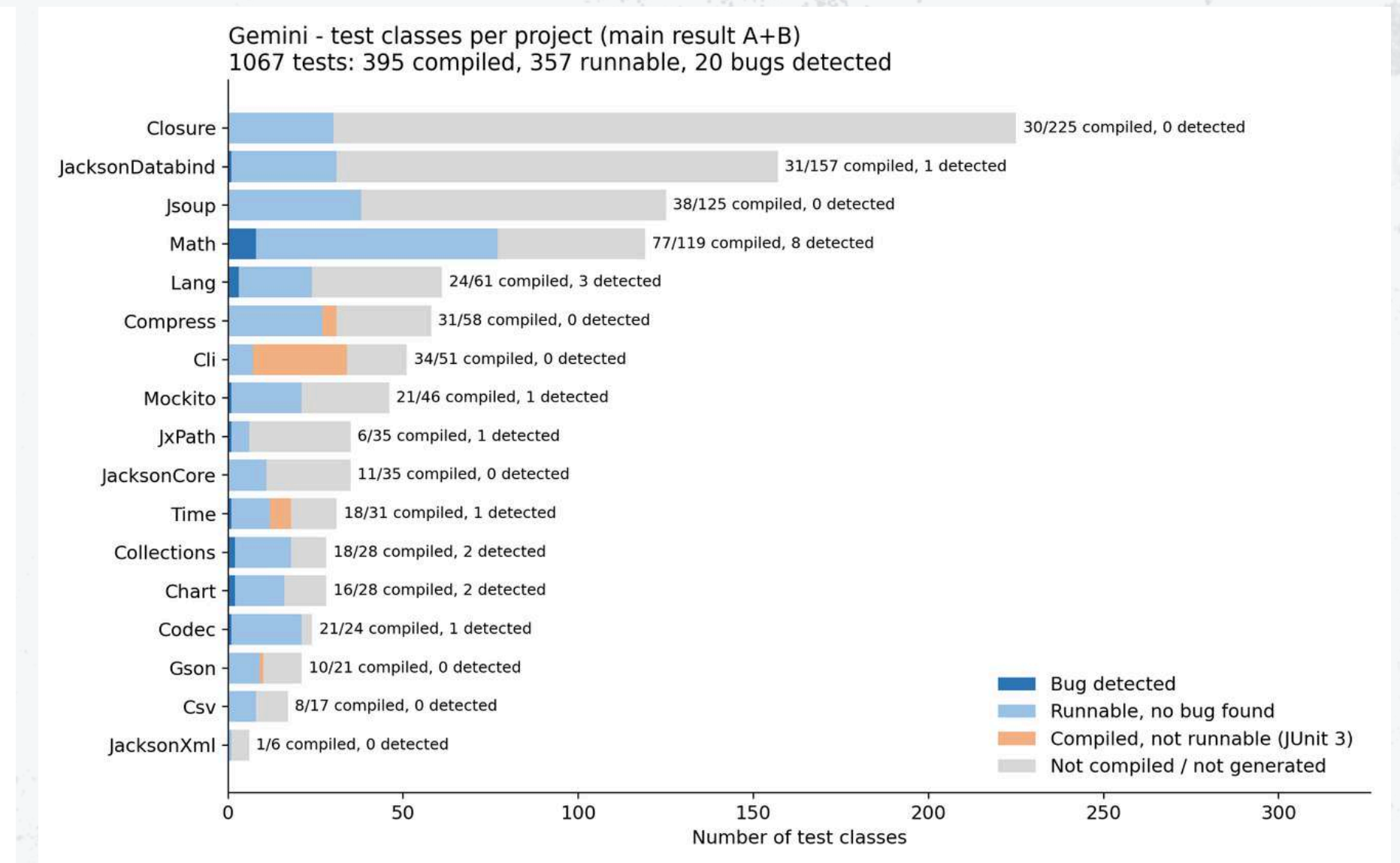
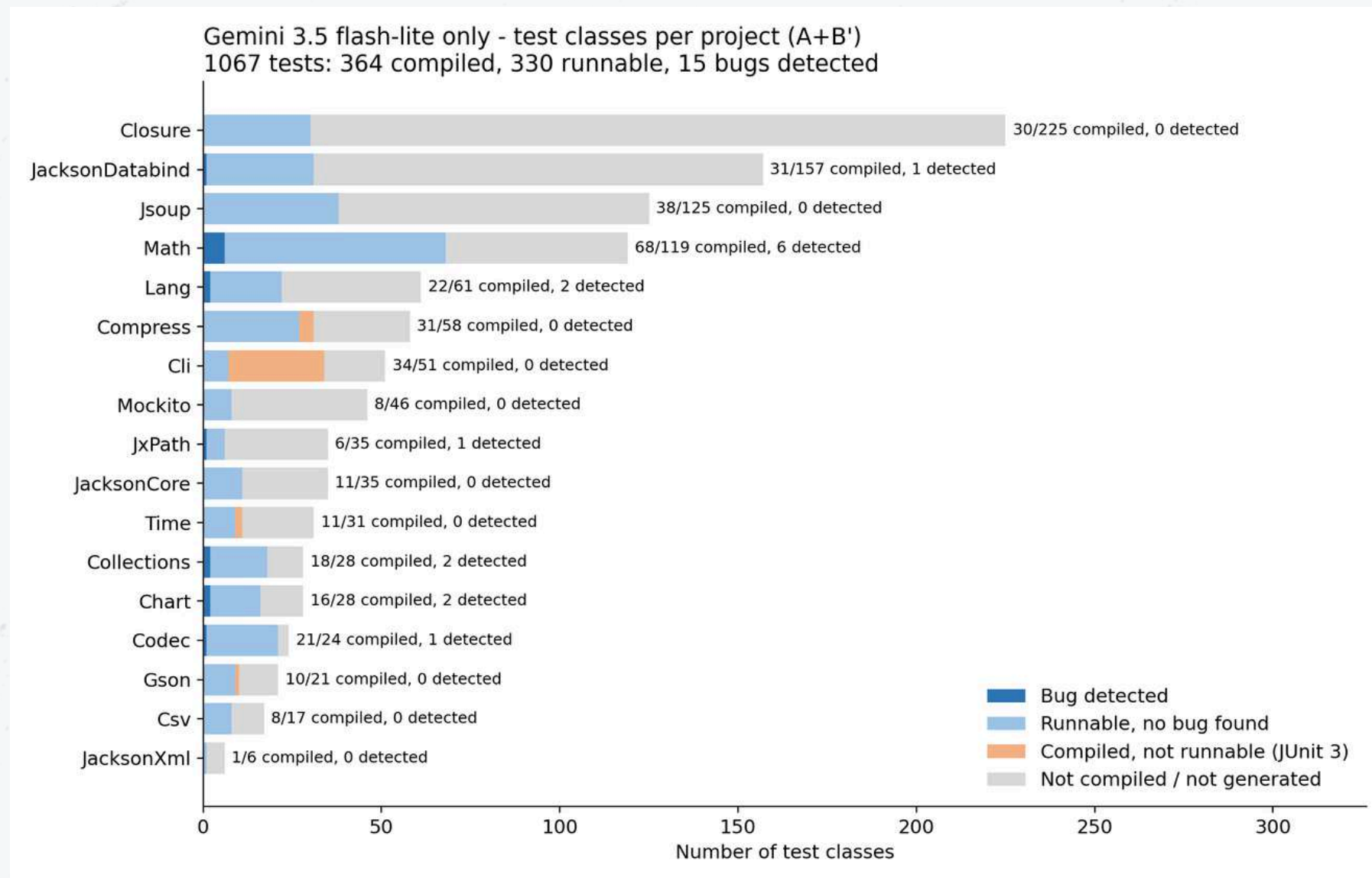
- N ของ coverage (ผสม) = 392 test — นับต่อ test เพราะบาง "บ๊ัก" มีหลายคลาส
- N ของ coverage (flash ล้วน) = 361 test
- N ของ FDR = 854 บ๊ัก



- ในที่ Complie ที่ไม่ได้ผลนั้นคือ JUnit เวอร์ชันเก่า ไม่ใช่ JUnit4 จึงรันไม่ได้นั่นเอง

RESULTS

19



3.5 flash-lite ล้วน – จับได้ 15

ผสม 3.5 + 3.7 – จับได้ 20

- Math จับบักได้มากที่สุด (6 → 8) เพราะเป็นฟังก์ชันคำนวณ ทดสอบค่าขอบได้ตรง
- Closure compile ผ่านน้อยสุด (30 / 225) โค้ดซับซ้อน AI เดา API ผิด
- Mockito ต่างกันมากที่สุด: รุนเล็ก compile ผ่าน 8 รุนใหญ่ 21 จาก 46

COMPARISON

- 3.7 flash: compile ผ่านมากกว่าถึง 24 บั๊ก และจับบั๊กได้เกือบ 2 เท่า (11 กับ 6)
- ต่างมากที่สุดที่ Mockito: compile 19 กับ 8 บั๊ก รุ่นเล็กมีปัญหาเกี่ยวกับโปรเจกต์ที่ API ซับซ้อน

| เปรียบเทียบ configuration บน 192 บั๊กชุดเดียวกัน (B กับ B') | | |
|---|----------------------------------|--|
| ใช้บั๊ก, prompt และวิธีวัดชุดเดียวกัน ต่างแค่รุ่นโมเดล | | |
| ตัวชี้วัด (ต่อบั๊ก, 192 บั๊ก) | B: <code>gemini-3.7-flash</code> | B': <code>gemini-3.5-flash-lite</code> |
| compile ผ่านอย่างน้อย 1 คลาส | 106 (55.21%) | 82 (42.71%) |
| บั๊กที่จับได้ | 11 | 6 |
| Fault Detection Rate | 5.73% | 3.12% |
| FDR เฉพาะบั๊กที่ compile ผ่าน | 10.38% | 7.32% |
| Line / Condition Coverage (เฉลี่ยต่อคลาส) | 94.78% / 89.92% | 92.33% / 85.59% |

| โปรเจกต์ (192 บั๊ก) | บั๊ก | compile ผ่าน 3.7 / lite | จับได้ 3.7 / lite |
|---------------------|------|-------------------------|-------------------|
| Lang | 37 | 6 / 4 | 1 / 0 |
| Math | 91 | 64 / 59 | 8 / 6 |
| Mockito | 38 | 19 / 8 | 1 / 0 |
| Time | 26 | 17 / 11 | 1 / 0 |

SUMMARY

21

342/366

Executable Bugs

15/20

Kill Bugs

90.46/91.33%

Line Coverage

Runtime & Resource Usage

Total Generation Time: 11.4 ชั่วโมง (จาก Timestamp ของไฟล์)

Generation Attempts: 1067 Classes

Total prompt char: 36,156,994 ≈ **9,039,248 tokens (ONLY FLASH 3.5 lite)**



Claude

CLAUDE

Model

Claude sonnet 5

COVERAGE

614/854 → (71.90%)

Kill_bugs

46 / 854 → (5.39%)

PROMPT

คุณคือ Java testing engineer ที่เชี่ยวชาญด้าน unit testing และ JUnit 4
เป้าหมาย: เขียนชุดทดสอบ JUnit 4 สำหรับคลาสด้านล่าง เพื่อให้ได้ branch coverage สูงที่สุดเท่าที่เป็นไปได้

และมีโอกาสดักจับข้อบกพร่อง (fault) ในโค้ดได้จริง

ข้อกำหนด:

1. ใช้เฉพาะ JUnit 4 และไลบรารีที่อยู่ใน classpath ต่อไปนี้: {cp.test}
2. ตั้งชื่อคลาสทดสอบเป็น {ClassName}Test และ import คลาสเป้าหมายให้ถูกต้อง
3. ครอบคลุมกรณี: ค่าขอบเขต (boundary), ค่า null/ว่าง, อินพุตผิดปกติ, และเงื่อนไข if/else, loop ทุกสาขาเท่าที่วิเคราะห์ได้จากซอร์สที่ให้มา
4. ห้ามเดา behavior ที่ไม่มีอยู่ในซอร์สโค้ด หากไม่แน่ใจให้เขียนคอมเมนต์กำกับ
5. หลังโค้ดทดสอบ ให้สรุปเป็นตารางสั้น ๆ ว่าแต่ละเมธอดทดสอบใด ครอบคลุม Branch/Condition ใด

ซอร์สโค้ดคลาสเป้าหมาย (Defects4J: {project}-{bug_id}b):

```
```java
{source_code}
```
```

RESULTS

24

Average Compile rate 73.7 %

Average Line coverage 81.14 %

(n = 1055)

Kill Bug 46 / 854 → (5.39%)

Claude Sonnet 5 API Token PriCing

Input \$2 / MTok

Output \$10 / MTok

Token Usage

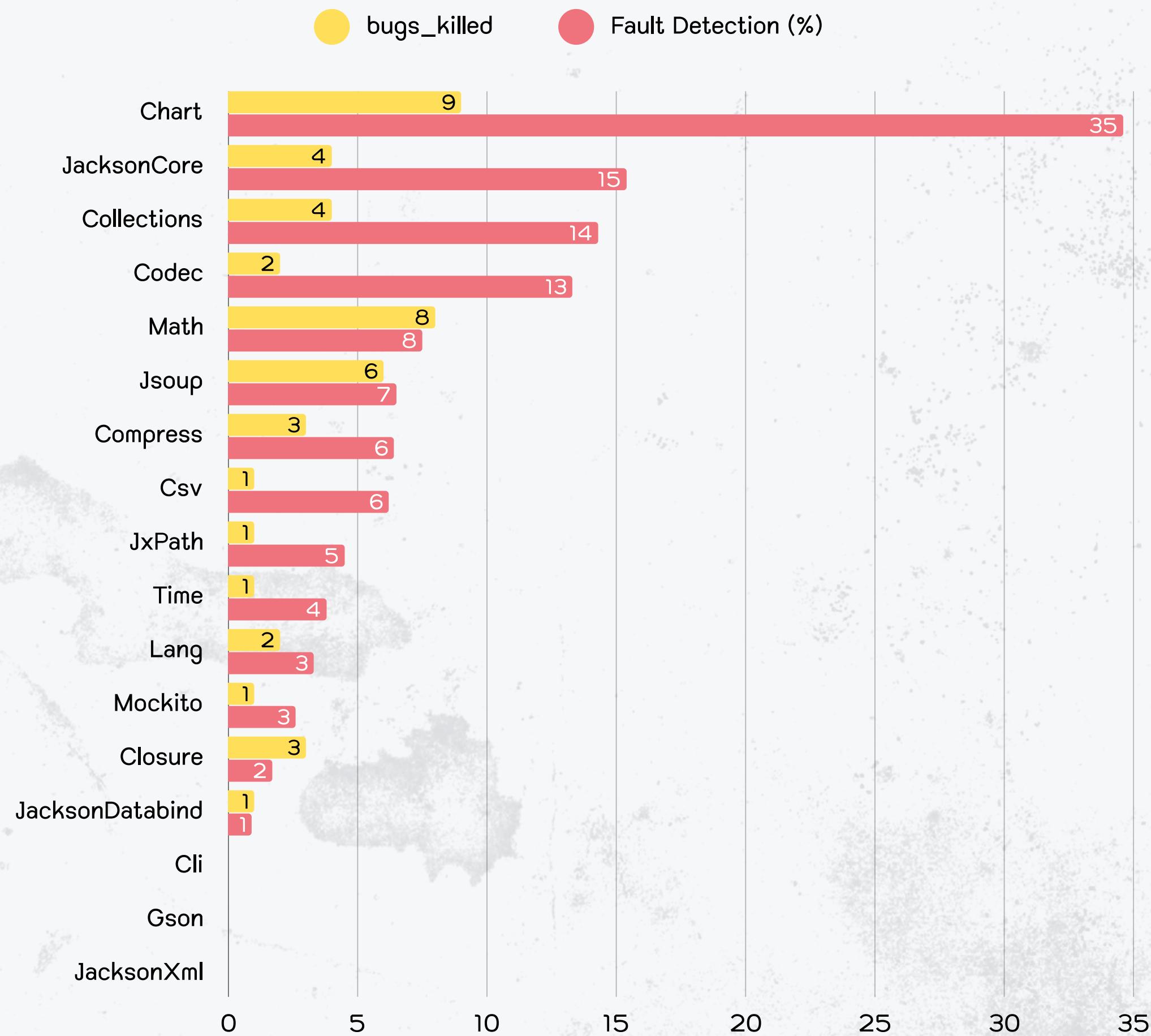
Total Input Token

12,468,957

Total Outout Token

34,776,988

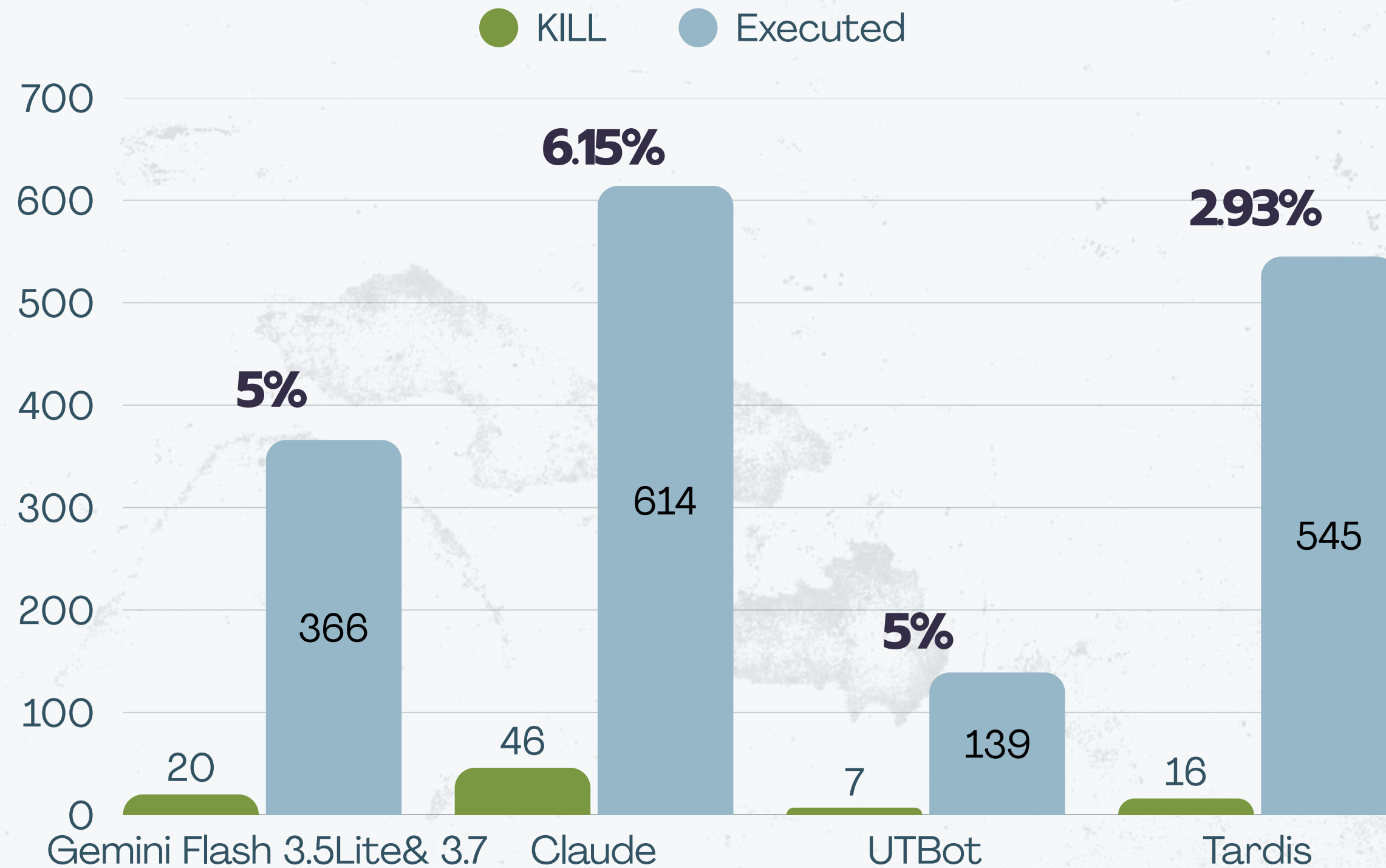
Total = 47,245,945



Compare the totals

Executability & Fault Detection

26



Summary

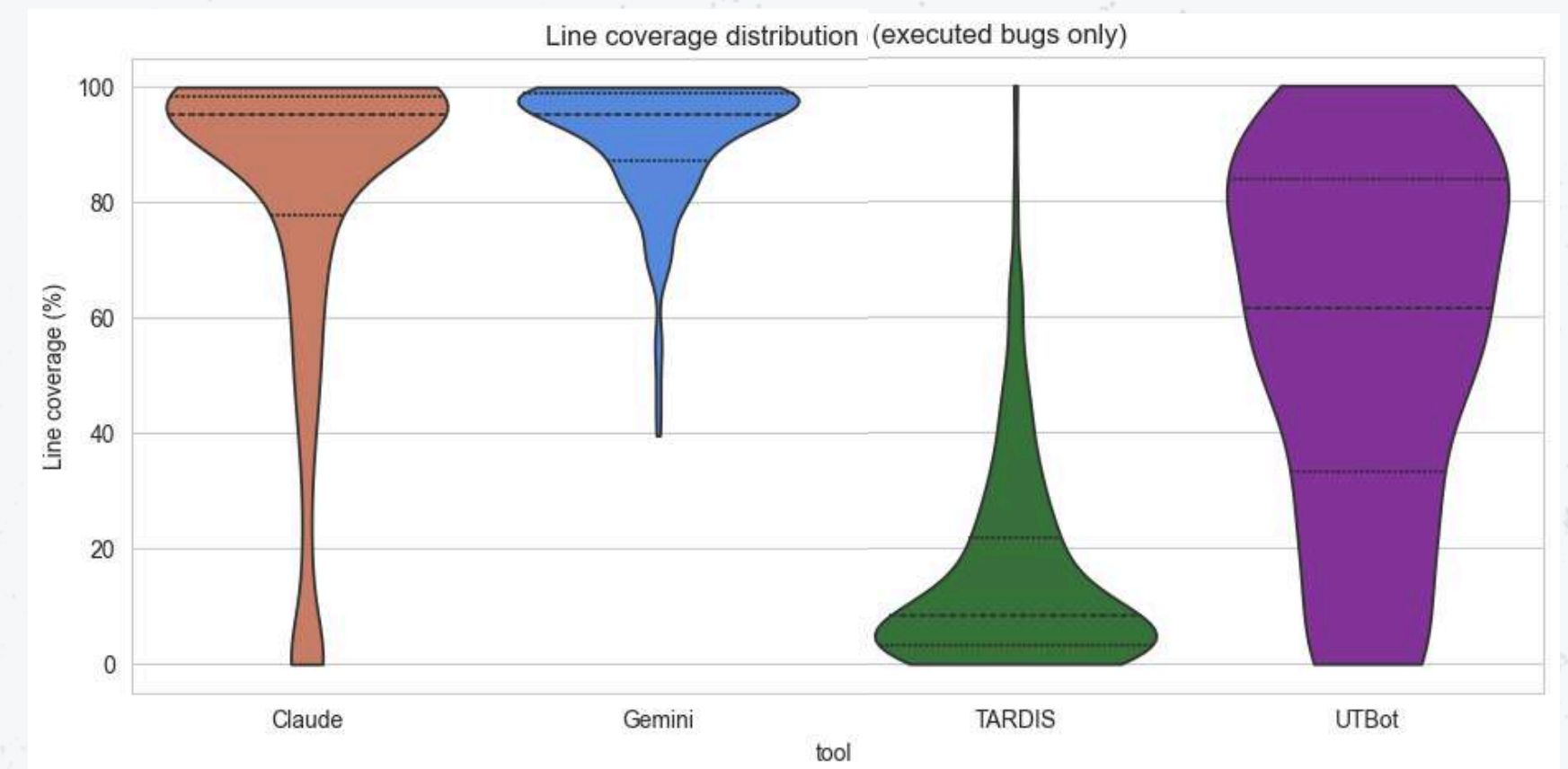
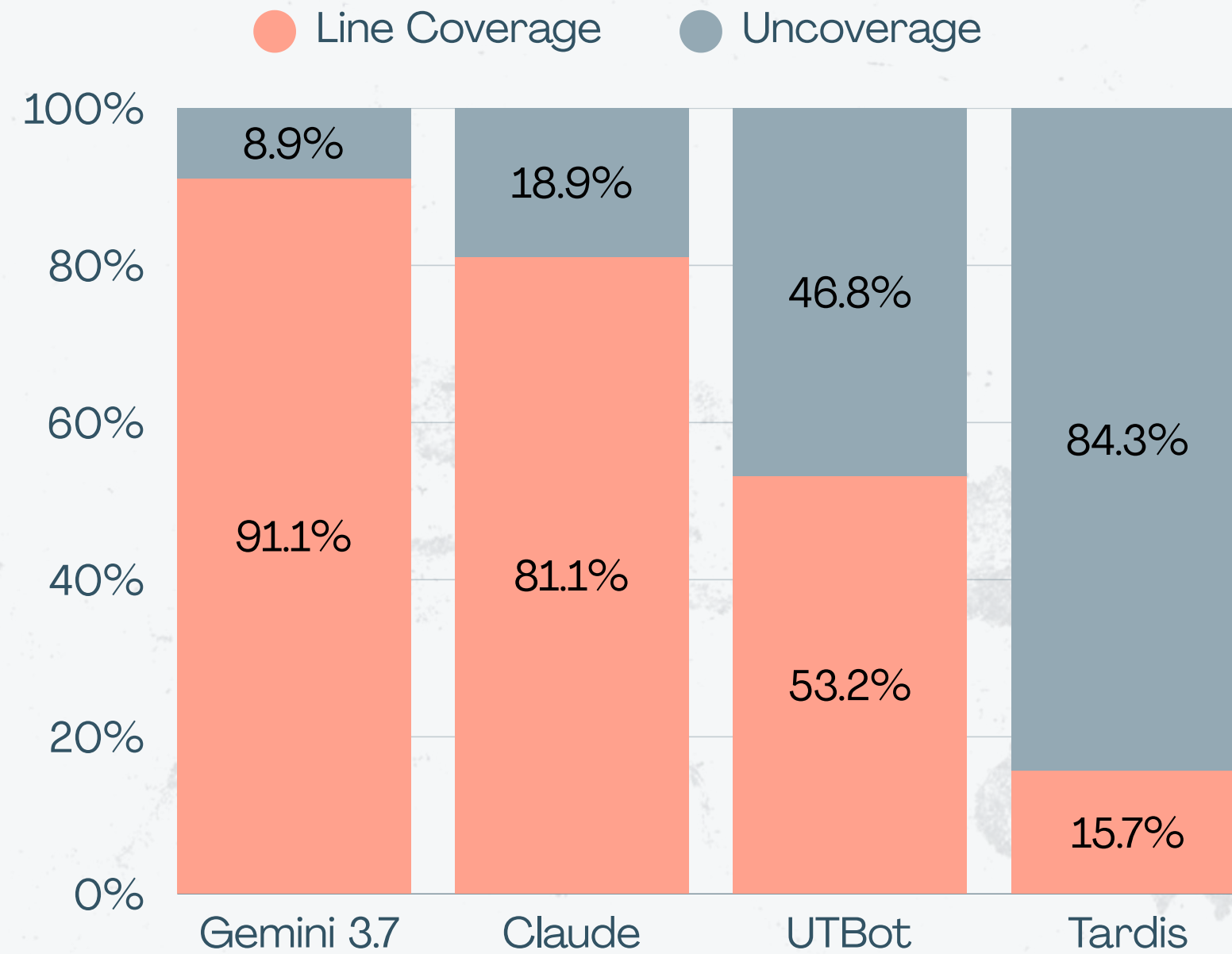
Executed Rate สูงที่สุดคือ Claude ตามลงมาด้วย Tardis และมี การ Kill ต่อ Executed มากที่สุดคือ Claude รองลงมาคือ Gemini และ UTBot

หาก UTBot สามารถ Fuzzing ได้ อัตรา Kill Bug ก็จะสามารถเพิ่มขึ้นได้

Compare the totals

Code Coverage

27

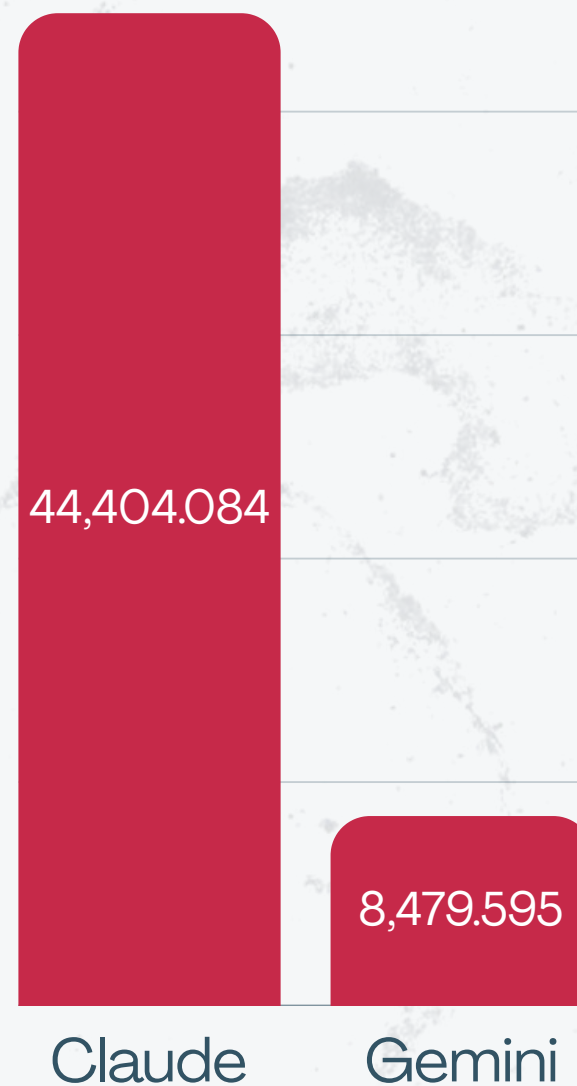


Compare the totals

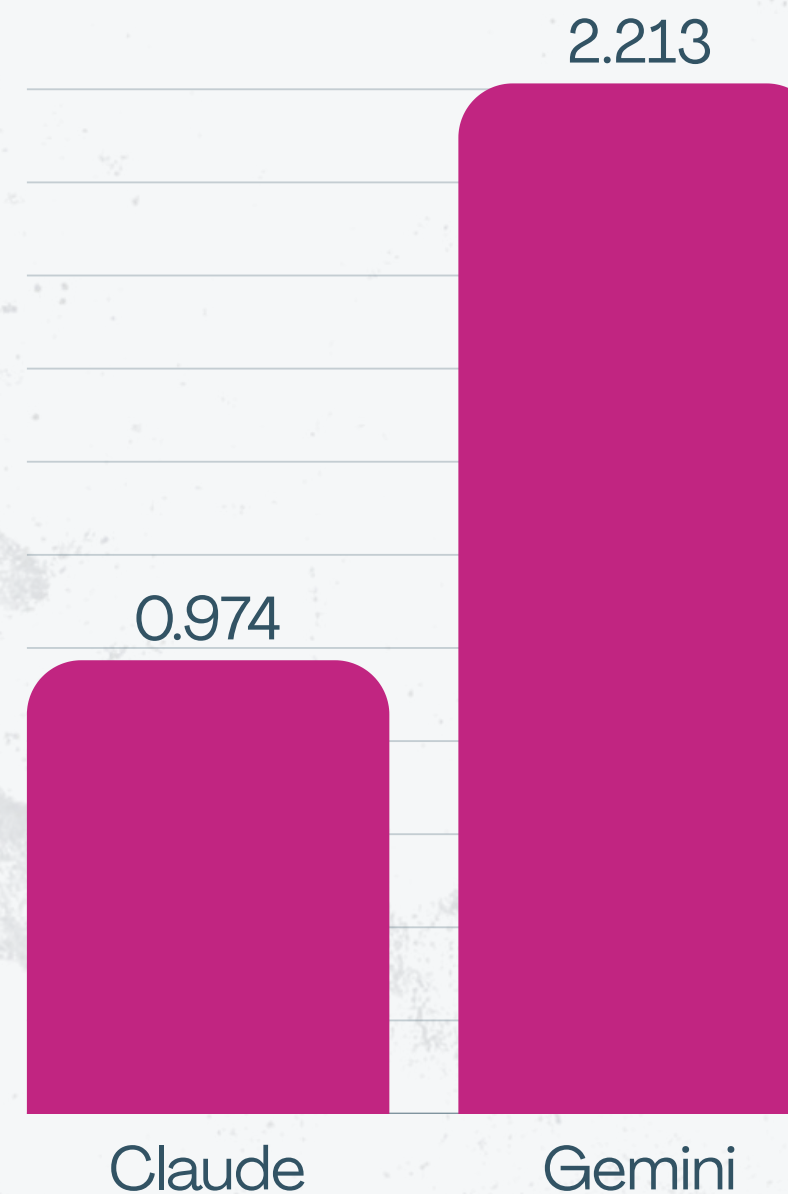
Token Resource

28

Average Token Spend
Per Bugs



Killed Bugs Per 1M Tokens



Summary

Claude สามารถ ทำ executable rate และ kill ได้เยอะ แต่ ก็แลกมา กับ Token จำนวนมากที่ต้องจ่าย เท่าตัว

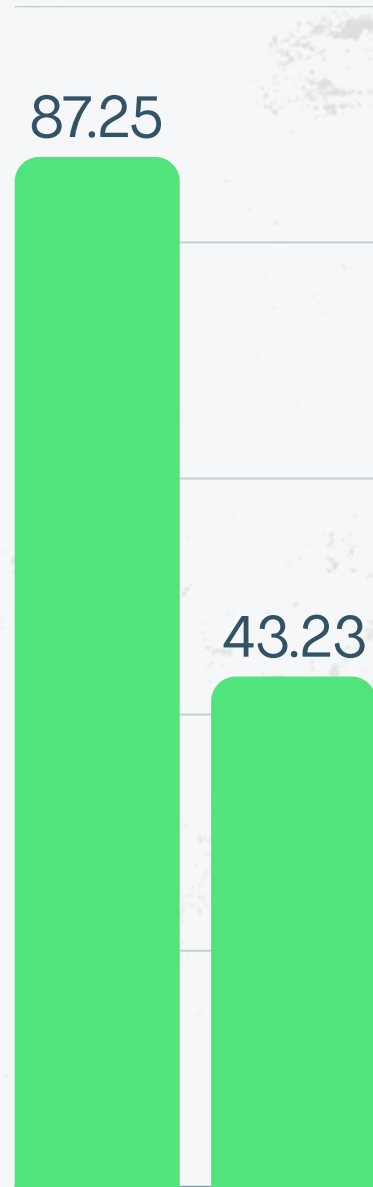
Gemini ไม่มี token log จึง ประมาณ input tokens จาก `prompt_chars / 4` (โค้ด Java $\approx$ 4 ตัวอักษร/token)

Compare the totals

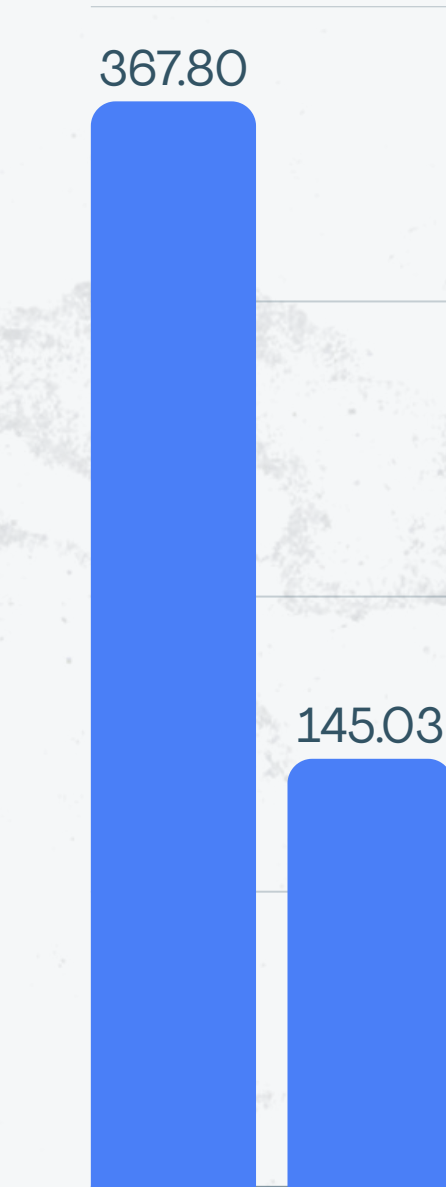
Time Resource

29

total_gen_hours



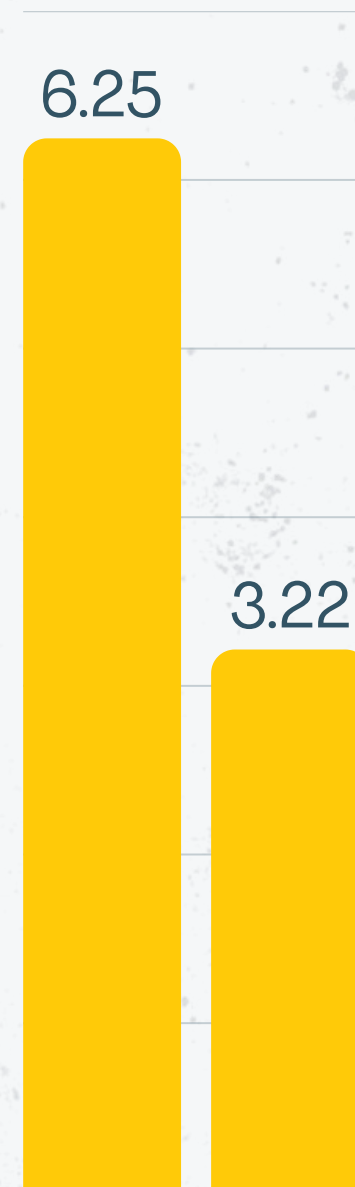
mean_sec_per_row



killed_per_hour



executed_per_hour



Summary

เห็นได้ชัดว่า TARDIS ต้องการ Resource ระยะเวลา เยอะกว่า UTBot แต่สามารถ ทำ ผลงาน executable rate และ kill ได้เยอะกว่า UTBot

Compare the totals

Time Resource

30

| Model/Algorithm | Senario | Pros | Cons |
|-------------------|---|--|---|
| Claude | มีงบ API, ต้องการ Execute Rate เยอะที่สุด | executable rate และจำนวน kill สูงสุด
เข้าใจเป้าหมายของโปรแกรม , Assert จากความหมายได้ | ใช้ Token มหาศาล / เสี่ยง Data Leakage ไม่เหมาะกับองค์กรระดับ Enterprise |
| Gemini Flash-lite | งบ API น้อย, ต้องการ coverage | ต้นทุน token ต่ำ / Coverage ของ test รับได้สูง | Executable Rate ต่ำกว่าโมเดล Claude / Coverage สูงแต่ assert ตื้น / เสี่ยง Data Leakage ไม่เหมาะกับองค์กรระดับ Enterprise |
| TARDIS | มี Resource RAM และ เวลา รับเข้ามาคืบ | ไม่เสี่ยงข้อมูลรั่วไหล, Executable Rate สูง , จับ Exception และ Mathematic ได้ดี | ใช้ทรัพยากร Ram และ เวลา สูง / Coverage ต่ำ / ไม่เหมาะกับการตรวจ Logic Bug / ไม่เข้าใจเป้าหมายของโปรแกรม |
| UTBot | ต้องการความปลอดภัย งบ น้อย ไม่มี Dependency ซับซ้อน | Kill bugs rate เทียบกับเวลา และ Executable rate แล้ว อยู่ในระดับที่ประหยัด / Coverage ค่อนข้างสูง / เหมาะหากเจอ โปรแกรมที่เป็นระดับ Utility ไม่มี Dependency ซับซ้อน | ไม่เหมาะกับ Project ที่มีความซับซ้อนสูง / ไม่เหมาะกับการตรวจ Logic Bug / ไม่เข้าใจเป้าหมายของโปรแกรม |

DEMO

THANK YOU

SO MUCH!